Railway scheduling

Nils Bröcker, Julian Bruns
nils.broecker@uni-potsdam.de
julian.bruns@uni-potsdam.de

University of Potsdam
Institute of Computer Science



Abstract. In this paper, we address the complex challenge of efficiently managing traffic within railway networks, a task complicated by numerous physical constraints inherent in real-world systems. We utilize the 'Flatland' framework, which provides a simplified model for railway scheduling problems, allowing us to focus on certain key aspects. The Flatland instances are structured as finite 2D-grids where trains navigate in discrete time steps, adhering to specific railway statements that dictate their movements. Each train has a designated goal station, and our objective is to ensure that all trains reach their goals without collisions, even in the presence of potential delays.
We implement various approaches to solve these problem instances using the Answer Set Programming tool 'Clingo', exploring the possibilities of single- and multi-shot approaches. We conduct testing on generated training instances to evaluate the performance of the presented and implemented approaches. The results of our benchmarks provide insights into the effectiveness of each method, leaving space for a lot of conclusions and further research in the future.

1 Introduction

A major problem in the world of transportation is the task to efficiently manage dense traffic on complex railway networks. Since real world railway networks have a great number of physical constraints that makes tackling such a problem even more difficult, it is a good idea to try to simplify this problem first. One such Simplification is the so called "Flatland" framework [Fla]. We try to solve problem instances in this framework with the help of the Answer set programming tool "Clingo".
In this short paper we will present multiple different implementations, discuss their benefits and flaws and test them on generated training instances.

2 Flatland framework

To solve the question how to efficiently manage dense traffic on complex railway networks, the Flatland challenge provides a simplified framework. This framework describes how problem instances of railway networks and their solutions are structured.

Flatland instances consists of a finite two-dimensional grid where every cell can hold a single train, which can move in discrete time steps. The movements that trains are allowed to perform are dictated by the direction of the train and by one of several possible railway statements that are located on each cell. Each train can have exactly one of the four cardinal directions, north, east, south, and west. Additionally there are goal stations for each train that are located on certain cells, which the trains need to reach at some timestep. Lastly, in each timestep there is a possibility of a delay occurring. Such a delay prevents a single train from moving for a specific number of time steps. The goal of these Flatland instances is to move each train to their corresponding goal station, without causing any collisions, even when delays occur.

2.1 Fladland structure

The official Flatland documentation can be accessed here. We however use a simplified version of Flatland that has slightly modified structure.

Structure of problem instance Our Flatland problem instances consists of four different types of statements:

- cell((Y,X), ID) statements that describe the railways that are on each cell, where the *ID* is the specific type of railway and *(Y,X)* describe its X and Y position. In our case we can ignore cells that have an *ID* of 0, since they represent cells without any railway, which means no train could reach them in the first place.
- agent(ID) statements that simply describes how many trains we have and what *ID*s they use. For n trains we usually have trains with *ID* ranging from 0 to n-1.
- start(agent(ID),(Y,X),T,dir(D)) statements that describes, where, when and in what direction each train starts. The ID is simply the train *ID*, *(Y,X)* its starting position, T its starting time and D its starting direction.
- end(agent(ID),(Y,X),T) statements that describe where and when each train has to reach their specific goal. *ID* and *(Y,X)* represent the train *ID* and its goal position and T represents its latest arrival time. It may reach its goal sooner but not later than this time.

Note, that the grid size is technically not specified directly by any statements but since there are a limited number of cells, the cell size is implicitly given as well.

Structure of results The results on the other hand only consist of one type of statements, the action(agent(ID),Type,T) statements that describes the type of movement each train carries out at a specific timestep. Here *ID* and T likewise stands for the train ID and the time that the action occurs. The *Type* on the other hand describes what kind of movement the train performs. There are four different types of movement options:

- *wait* option, which simply says that the train has the same position and direction in the next timestep.
- *move_left* option, which means that the train goes to the cell that is to the left of its current direction and rotates its direction to the left. So if a train would look toward the East and use the *move_left* option, it would look to the north and also go to the northern cell in the next timestep. This option can only be used, if the train has an option to move to the left or forward/to the right. If this is not the case and there is only one option, we would use the *move_forward* option.
- *move_right* option is equivalent to the *move_left* option, just with movement and rotation to the right and not to the left.
- *move_forward* option is used to move the train to the cell in its direction if there is a choice. If there is no choice and only one option to move either forward, left or right, then the *move_forward* is still used, even if the only choice is to move left or right.

3 Implementations

3.1 Naive Multi-shot-solving

In our first working implementation, we use a simple multi-shot-solving program to iteratively search for a satisfiable solution. The program code can be found here. With multi-shot-solving we can use a python program to iteratively call Clingo to try to solve the problem instance. In each iteration we increase a specific variable, in this case the maximum time, until we find a satisfiable solution. Our implementation can be divided into three program parts denoted by the *#program* statements.

Program part: base The first program part, called **base** is only ever grounded once and contains the static knowledge for every iteration. Here, we encode the behavior of the cell Types into *track* statements. Each cell *Type* represents a certain track type and therefore dictates how a train can enter and exit this cell type. Each track((Y,X),DirIn,DirOut) statements consist of the position of the specific cell and the possible incoming and outcoming directions.

The four cardinal directions starting from north and going clockwise, are represented as numbers going from 0 to 3. So a 0 means north, number 1 means east and so on. So, as an example, the statement track((1,1),2,0) would mean that a train could move into the cell at position (1,1) from the south and then move out of the cell to the north. Since train tracks are always bi-directional the cell would also have a ((1,1),0,2) since trains could come from the north and leave through the south. Each possible train type is encoded in such a way, with the only exception being that we don't encode the cell type 0. This is because they represent cells without any train tracks which are cells that a train could not enter in the first place.

After that we encode a few additional statements that help with the representation and handling of directions:

- The *dirConv(letterDir,NumDir)* statements simply help transform the letter representation of direction ("n","e","s" and "w") that the problem instances uses into number representation, that we find a bit more easy to work with.
- The dirMove*(Dir,(Y,X))* statements represent the four cardinal directions in cell movements as *(Y,X)* vectors. As an example the direction north would be represented as (-1,0), since going one step north means reducing your y-position by one.
- The revDir(Dir,DirRev) statements are used to reverse directions. This is useful since leaving a cell from a direction (for example north), means entering the next cell from the opposite direction (e.g. south).
- The rotLeft(Dir,DirLeft) and rotRight(Dir,DirRight) statements both simply rotate their direction, which is useful when a train turns toward the left or the right.

Next we encode the starting positions of the trains. Our train positions are represented with train(*ID,(Y,X),Dir,T*) statements. That encode the ID, the position, and the direction of the train in the timestep *T*. Since the start statements are very similar, we just need to copy the ID, position and *EarliestDeparture* time and then convert the letter representation of the direction into a number representation.

Lastly we create singleChoice*(Y,X,DirIn)* statements that represents in which cell and with what incoming direct a train would have no choice between moving forward or left or right. This is necessary since in such situation a train would always need to wait or move forward, even if the direction of the train would change. All of this represents our **base** knowledge. Next we go to the **check** program which represents constraints that are only valid in one single iteration but not anymore in later iterations.

Program part: check The first constraint in the **check** program is the goal constraint. It makes sure, that every train ends up at their goal location some point before or at the maximum time but not later. Then we also have the Latest arrival constraint that make sure, that each train also reaches their goal before their specific latest arrival time.

Lastly we have the **step** program part, that represents knowledge that accumulates in each iteration.

Program part: step The first thing we do in the step program part is to handle train movements. Here we implement the rule that every train statement has a corresponding train statement in the next timestep that either waits or moves along the current train track. The only exception to this rule are trains that are in the maximum timestep.

We furthermore add two constraints that prevent collisions. The vertex constraint prevents two different trains from occupying the same space at the same time and the edge constraint prevents two trains from moving through each other.

Next we take successive train statements and create the corresponding action statements, which as we discussed earlier are the statements that we need for the output. Here we also use the singleChoice statements to differentiate between some of the movement types.

Lastly we implement the delays. We handle delays with delay(ID,T,Dur) statements, where the *ID* and *T* says what train and at what timestep a delay occurs and *Dur* says how long the delay occurs. Such a delay statement simply makes sure that the current and next *Dur-1* train actions will have the type *wait*.

Flaws Unfortunately this implementation still has a big issue. While it technically give a correct answer at the end, the solving process is not very realistic, since it can simply change previous train/action statements if a new delay occurs. This is obviously not possible in real live, so we need an approach that makes sure that train/action statements in previous timesteps don't change anymore. Though this implementation is still very useful as a base for our next implementations.

3.2 Two-step Multi-shot-solving

To solve the aforementioned issue of retroactive changes to the train statements in previous timesteps we transformed our implementation into a two-step approach. In the first step we generate a *baseplan*, that does not handle delays at all. This *baseplan* is generated in almost the exact same way that the naive Multi-shot-solving plan is generated, just without generating and dealing with delays at all. So if no delays would occur, this *baseplan* would be an optimal solution. But with delays we can still use this *baseplan* as a template. We use the *baseplan* as an additional input and then execute another Multi-shot-solving file that uses the *baseplan* statements as instructions. More concretely this means that all train statements of previous timesteps corresponds to one specific *baseplan* statement. However, when a delay occurs all other trains have the option to wait, in which case their "corresponding *baseplan* statement" is moved back. If a train has been delayed in total by 2 timesteps their corresponding *baseplan* statement would be 2 timestep behind. Since the *baseplan* does not change and delays can only appear in the current iteration, not in previous iterations, the train statements in previous timesteps also never change. This solves the issue of the Naive Multi-shot-

solving implementation

Ideally if a delay occurs that makes sure, that a train is 4 steps behind its *baseplan*, then we want to give every other train the option to wait as long as they are not longer then 4 steps behind their *baseplan*. In the worst case scenario this would mean, if a train has a total delay of 4, every other train would also wait a total of 4 timesteps. However, actually calculating how much the other trains could theoretically wait while not waiting longer in total then the original train is much to inefficient, so we simply say that every other train can wait as many timesteps as the current time. As an example, in timesetep 5 a train could have a maximum delay of 5, so every other train could also wait 5 timesteps. This approach ignores if the other trains already have some delays and allows for much longer waiting intervals but it is still much more efficient then the previous more optimal approach.

Additionally we need to replace the previous vertex and edge constraint, since the program might try to solve a vertex/edge constraint by letting a train wait that would ordinarily move first. This can destroy the order of trains and potentially lead to issues if one train has to move first through, for example, a narrow passage. Because of this we replace these constraints with a simplified *Temporal Plan Graph* (TPG) constraint. These make sure that the order in which each train enters a cell in the *baseplan* and in the current plan stays the same.

Lastly we still have the problem, that the this iterative approach does not differentiate between unsatisfiable because the trains are not on their goal or unsatisfaiable because the TPG constraint has been violated. Since the Multi-shot-solving approach tries to iterative solve the program as long as the result is still unjustifiable, is could be that the previous trains violated a TPG constraint, that can't be solved in future iterations. So to solve this, we need to execute the program once without the goal constraint so that trains are generated that do not violate the TPG and then in the same iteration we execute the program a second time with the goal constraints

enabled to make sure that the trains will reach their goal in some future iteration.

3.3 Minimized-delay Multi-shot-solving

The *minimized-delay Multi-shot-solving* is almost the exact same Implementation as the *two-step Multi-shot-solving*, just with the difference that an additional minimization statement has been added, that minimizes the amount of delays in each iteration. If a delay occurs and a train has to wait, this potentially means, that other trains have to wait and delay their movement in order to not cause a TPG violation. This optimization statement makes sure that in this iterations the number of other trains that have to wait is as low as possible. The hope is that this will cause for a low total amount of delays at the end and therefore make sure that the trains will reach their goal sooner. Optimizations statements are often very computationally intensive so we expect that the time it takes to find a result will increase but at the same time the number of timesteps until every train is at their goal should decrease in some instances.

3.4 Minimized-distance Multi-shot-solving

Lastly, we have the *"Minimized-distance Multi-shot-solving"* implementation. This is another variation of the *Two-step Multi-shot-solving* implementation. However in this version we minimize the distance the train statements have to their goal. Furthermore we replaced the TPG constraint with simple vertex and edge constraints. The reason for this is that the TPG constraint enforces a strict order in which trains can move through some nodes. While this makes sure, that the trains follow the baseplan very closely, it can often lead to results in which the trains take longer to reach their goal. By replacing these constraints with simple edge and vertex constraints we hope to decrease the time it takes the trains to reach their goal, at the cost of no longer being able to solvable 100% of the problem instances.

4 Testing

After successfully implementing all our approaches, we went on to test them. Our goal was to test them for computation time, thus we decided to simply run the approached on all of our test instances to have some sort of comparability. To also be able to compare a little better, we decided to add a variable $self.randomSeed = 666$ which acts as a seed for the random generated delays. This does not apply for our first single-shot approach as that one does not handle delays. For the single-shot approach tho we had a unique variable that we had to set only there, which was the horizon that was set to $Horizon = 200$. The multi-shot approaches do not have a fixed horizon, as they incrementally increase it. The single-shot horizon was chosen as such after some previous testing to guarantee that no instance will stay unsatisfied within the time limit just because of the horizon. We decided to set it to $Timeout = 600$s.
For our multi-shot approaches we still had two variables to be set. First was the delay probability, which determines the probability of each agent to get a random delay each step, which we set to $self.delayProb = 0.01$. Second was the standard deviation of the delays, which we decided to set $self.standardDiv = 5$.

4.1 Test instances

Our instances used for the benchmarks consist of 34 different pieces. The first four ones were course-given simple instances as seen in figure 1.
The other 30 instances were generated with a tool [IGT]. The agent numbers there range from 2 to 30, while the grid sizes range from 30x30 to 125x125. The types of instances vary and include 'normal', as well as dense and sparse instances. Dense in the manner of instances mean small grid size with high agent count, while sparse means large grid size and low agent count. In figure 2 one can see an example of instance 01, including 5 agents in a 40x40 grid. In figure 3 one can see an example of instance 12, including 3 agents in a 30x30 grid.

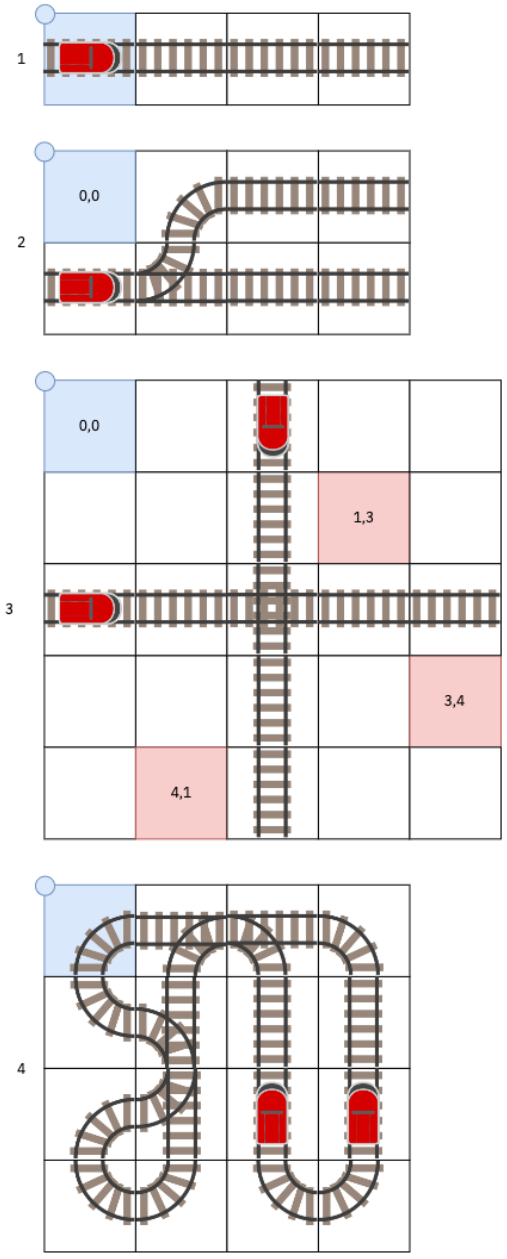


Fig. 1. Picture of the 4 basic instancs

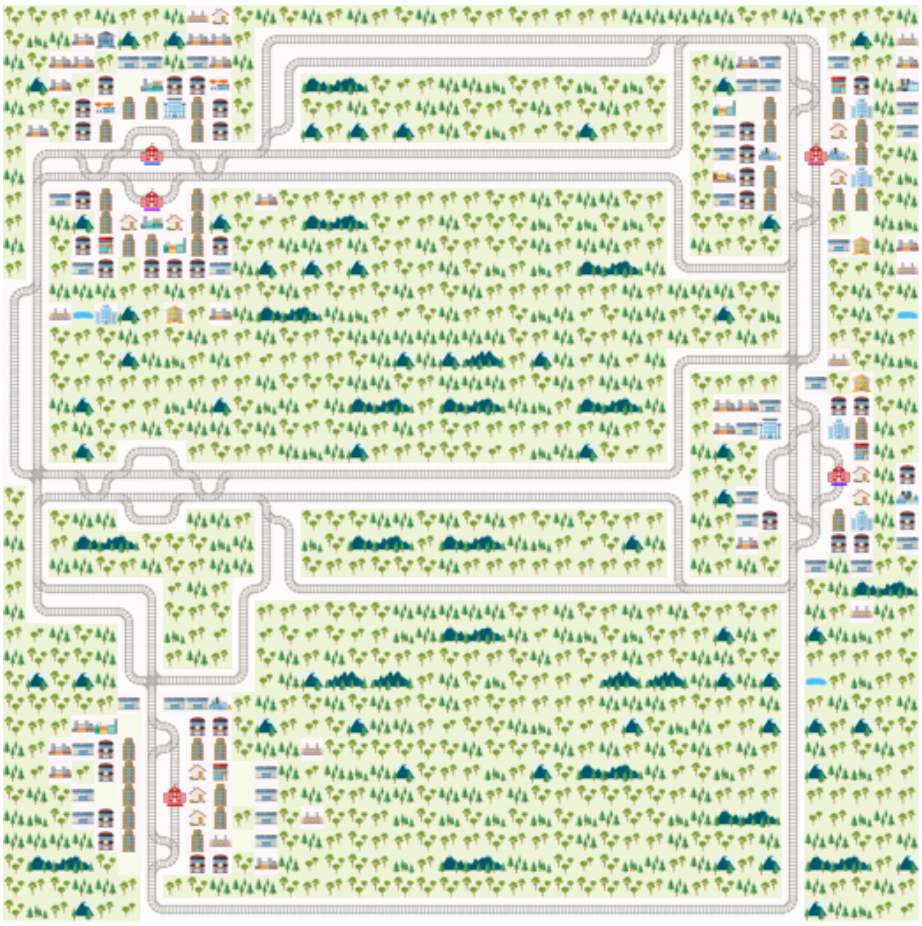


Fig. 2. Picture of instance 01



Fig. 3. Picture of instance 12

4.2 Tested approaches

In the next section there will be the benchmark results. There we will have numberings like "Benchmark 1". Now following is the list and explanation of benchmarked approaches.

- Benchmark 1 → single-shot approach
- Benchmark 2 → naive multi-shot approach
- Benchmark 3 → two-step multi-shot approach
  - Step 1 → baseplan generation (...MultiShot_init.lp)
  - Step 2a → plan execution and delays (...MultiShot_exec.lp)
  - (Step 2b → 2a but with agent delay minimization (..._exec_minDelay.lp))
  - (Step 2c → 2a but with edge/vertex conflicts instead of TPG and with minimizing the distance of agents towards their goal (..._exec_minDistance.lp))

The Step 2a only uses our first delay-handling approach, not the ones with minimized statements as described in sections 3.3 and 3.4. Benchmark 3 in the results later includes Step 2a, not b or c. To also compare these, we decided to compare only those three steps against each other, so Step_2a vs. Step_2b vs. Step_2c.

4.3 Results

In the following plots we visualized our benchmark results. The first plot in figure 4 is a full plot of our main 3 approaches, capturing all times all the way to 600 seconds, which was our timeout. Figure 5 is capped at $y_{max} = 25s$ to show the instances better that took way less time than 600s. Overall in both plots, it is clearly visible that our first single-shot approach with the blue bar is usually the slowest. And that while still not handling any kinds of delay. To explore that comparability a little we decided to plot the single-shot approach (blue bar) against the approach 3.1 (yellow bar), which is the first step of our two-step multi-shot approach that generates baseplans and does not handle delays yet, such doing a similar task as the single-shot approach. This plot also includes the naive multi-shot approach as it also is a single-step approach, but with delays. The final plot for this can be seen in figure 6. This plot clearly shows that the multi-shot approach of benchmark 3.1 is clearly superior to the approach of benchmark 1 in manners of computation time.

Figure 7 and 8 both show the plots of the three step 3.2 approaches plotted against each other. As they need baseplans to work on, they all got the same baseplans, that were previously generated by one benchmark of step 3.1 and they are also included in our repository. Figure 7 is already capped at $y_{max} = 350s$ as every benchmark that was $350s$ or more was always $600s$, thus having no value to plot it all the way to $600s$. As many instances are not visible in this plot because their computation times are too fast, we also did another plot that is quite fast capped at $y_{max} = 2s$, so only showing two seconds of computation time, but thus allowing to visualize the faster instances in a meaningful way. It is shown that approach 3.2a is usually faster than 3.2b or 3.2c, but it also does not include any minimize statement. The approach 3.2b which minimizes the times that non-malfunctioned agents are getting delayed by malfunctioning agents is often quite similar to approach 3.2a. Both implementations use the temporal plan graph concept, while approach 3.2c uses normal edge and vertex collision statements. Implementation 3.2c is often way slower than the other two approaches, whether this is due to the lack of TPG-using or due to the usage of a minimize statement that minimizes the remaining distance from each agent to their respective goal.

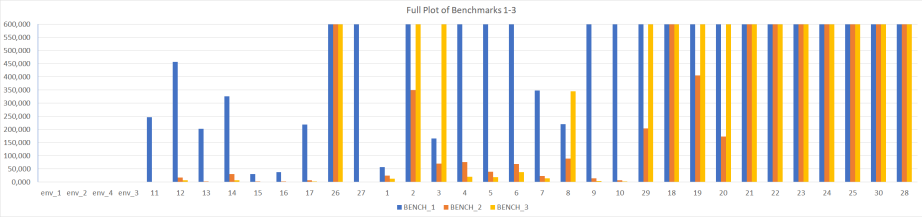


Fig. 4. Plot of Benchmarks 1-3

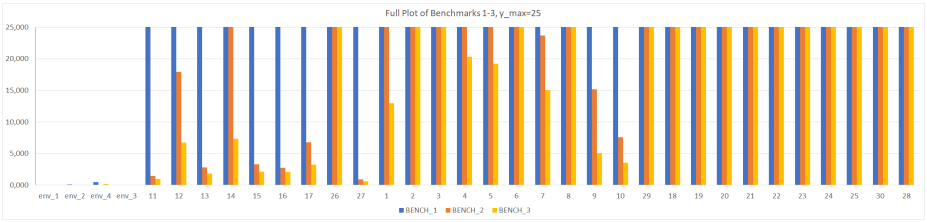


Fig. 5. Plot of Benchmarks 1-3, capped at y=25s

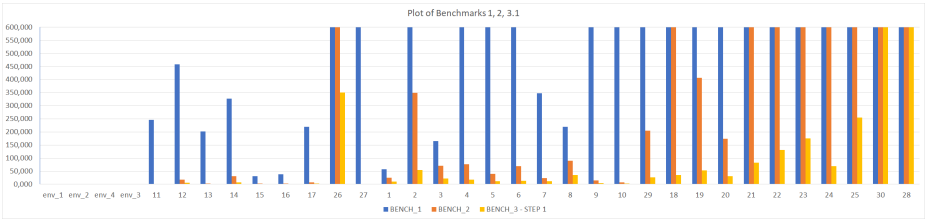


Fig. 6. Plot of Benchmarks 1, 2 and 3.1

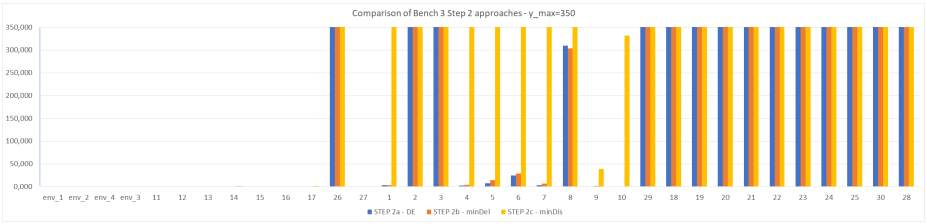


Fig. 7. Plot of Benchmarks 3.2a-c with $y_{max} = 350$s

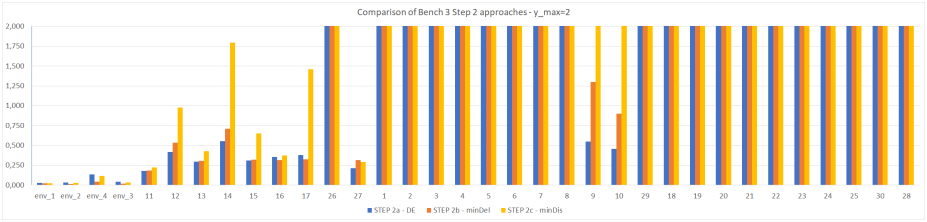


Fig. 8. Plot of Benchmarks 3.2a-c with $y_{max} = 2$s

5 Conclusion

After running, plotting and evaluating the benchmarks, there are a few statements that seem to be true. At first, multi-shot approaches here are generally faster than the single-shot approach. The two-step multi-shot approach that we implemented is usually the fastest but not always. On some instances the one-step multi-shot approach is faster, but that might be due to the fact that it is 'cheating'. By cheating we mean, that due to the single-step approach, the implementation can redo paths of agents in the past when a delay in the present step blocks the way long enough, that a different way would be shorter.
The implementation 3.2b that minimized the agent delays is faster than the implementation 3.2c, which minimizes the remaining distance to the agents goals. We assume that the main difference is the usage of TPG concepts in the 3.2b implementation, while 3.2c uses normal edge and vertex collisions. Thus we can conclude, that TPG implementations are faster than the some implementations with edge/vertex collisions regarding computation time. When looking towards the agents makespan, the TPG should not be optimal, as it ensures that every intersection is handled in the correct agent sequence, which can lead to a lot of unnecessary delays. However, we did not benchmark anything other than computation time, thus our conclusion also includes to test also different things than computation time next time.

References

Fla. Welcome to flatland. https://flatland.aicrowd.com/intro.html.

IGT. Flatland instance generation. https://github.com/krr-up/flatland/tree/main.